



CE 222 – Computer Organization and Assembly Language (COAL)

Lecture 31

Dr. Asad Mahmood

April 15, 2026

Lecture Outline

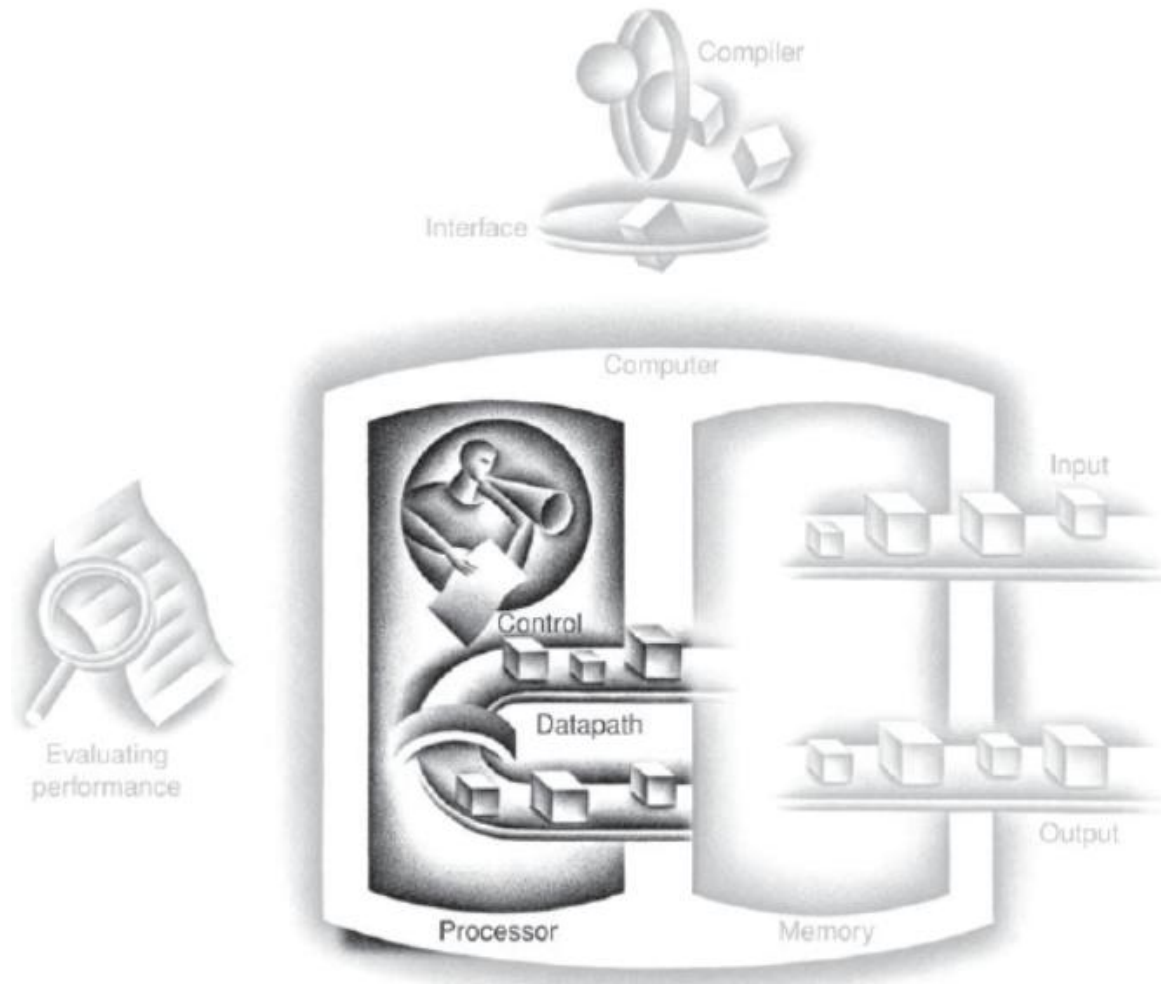
- Announcements (Mid re-checking, Assignment and Quiz 4)
- Outline of Previous Lecture:
 - **Chapter 4 – The Processor**
 - 4.5 Multicycle Implementation
 - 4.6 An Overview of Pipelining
- Outline of Today's Lecture:
 - **Chapter 4 – The Processor**
 - 4.6 An Overview of Pipelining

Chapter 4

The Processor

Chapter Theme

The Five Classic Components of a Computer



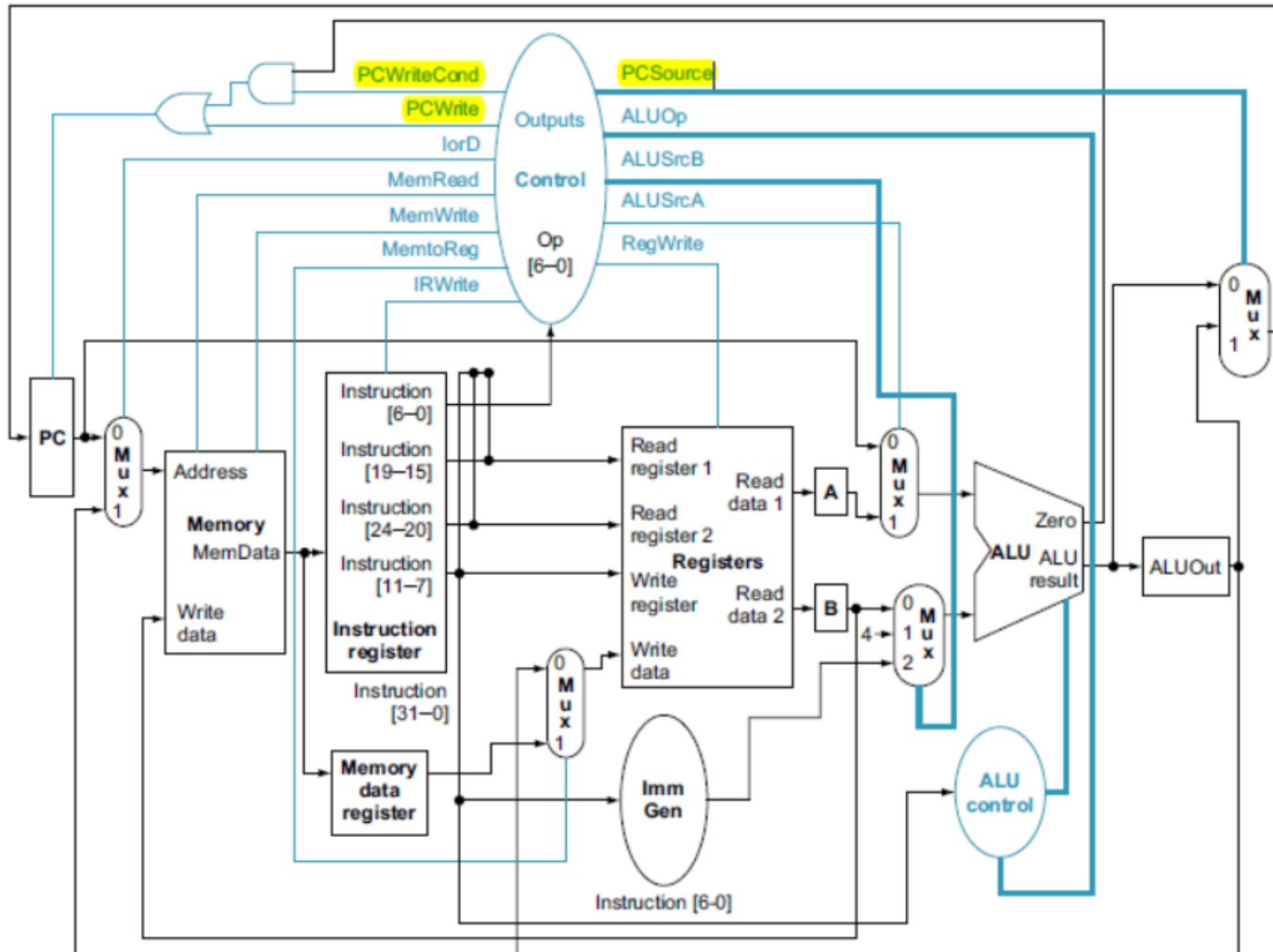
Chapter Outline

Chapter 4 – The Processor

- 4.1 Introduction
- 4.2 Logic Design Conventions
- 4.3 Building a Datapath
- 4.4 A Simple Implementation Scheme
- 4.5 Multicycle Implementation
- 4.6 An Overview of Pipelining
- 4.7 Pipelined Datapath and Control
- 4.8 Data Hazards: Forwarding versus Stalling
- 4.9 Control Hazards
- 4.10 onwards (self-reading)

Multicycle datapath

- Complete multicycle datapath including support for branch instructions



Multicycle datapath

Actions of the 1-bit control signals

Signal name	Effect when deasserted	Effect when asserted
RegWrite	None.	The general-purpose register selected by the Write register number is written with the value of the Write data input.
ALUSrcA	The first ALU operand is the PC.	The first ALU operand comes from the A register.
MemRead	None.	Content of memory at the location specified by the Address input is put on Memory data output.
MemWrite	None.	Memory contents at the location specified by the Address input is replaced by the value on the Write data input.
MemtoReg	The value fed to the register file Write data input comes from ALUOut.	The value fed to the register file Write data input comes from the MDR.
lorD	The PC is used to supply the address to the memory unit.	ALUOut is used to supply the address to the memory unit.
IRWrite	None.	The output of the memory is written into the IR.
PCWrite	None.	The PC is written; the source is controlled by PCSrc.
PCWriteCond	None.	The PC is written if the Zero output from the ALU is also active.

Actions of the 2-bit control signals

Signal name	Value (binary)	Effect
ALUOp	00	The ALU performs an add operation.
	01	The ALU performs a subtract operation.
	10	The funct field of the instruction determines the ALU operation.
ALUSrcB	00	The second input to the ALU comes from the B register.
	01	The second input to the ALU is the constant 4.
	10	The second input to the ALU is the immediate generated from the IR.
PCSrc	00	Output of the ALU ($PC + 4$) is sent to the PC for writing.
	01	The contents of ALUOut (the branch target address) are sent to the PC for writing.
	10	The jump target address ($IR[25:0]$ shifted left 2 bits and concatenated with $PC + 4[31:28]$) is sent to the PC for writing.

Multicycle datapath

Breaking the Instruction Execution into Clock Cycles

- Let us break instruction execution into a series of steps, where each step taken a single clock cycle
- This division is done such that major components (ALU, Memory, Register File) are only used once per step
- This also helps in keeping the clock cycle small

Multicycle datapath

Breaking the Instruction Execution into Clock Cycles

- Each RISC-V instruction needs from three to five of these steps:
 1. Step1: Instruction fetch step
 2. Step2: Instruction decode and register fetch step
 3. Step3: Execution, memory address computation, or branch completion
 4. Step4: Memory access or R-type instruction completion step
 5. Step5: Memory read completion step
- How many steps for R-type, memory-access, branch instructions?

Multicycle datapath

Step 1: Instruction fetch step

- Fetch instruction and load next instruction address in PC
- Generate appropriate control signals

Control Signal (1 bit)	Value
RegWrite	
ALUSrcA	0
MemRead	
MemWrite	
MemtoReg	
lorD	0
IRWrite	
PCWrite	1
PCWriteCond	

Control Signal (2 bit)	Value
ALUOp	00
ALUSrcB	01
PCSource	00

Multicycle datapath

Step 2: Instruction decode and register fetch step

- Act **optimistically** i.e. read registers and calc. branch address
- Generate appropriate control signals

Control Signal (1 bit)	Value
RegWrite	
ALUSrcA	0
MemRead	
MemWrite	
MemtoReg	
lrd	
IRWrite	
PCWrite	
PCWriteCond	

Control Signal (2 bit)	Value
ALUOp	00
ALUSrcB	10
PCSource	00

Multicycle datapath

Step 3: Execution, memory address computation, or branch completion step

- First step/cycle in which the action is instruction type-dependent
- R-type, memory-access and branch instruction action
- Following tables show control signals for the branch instruction

Control Signal (1 bit)	Value
RegWrite	
ALUSrcA	1
MemRead	
MemWrite	
MemtoReg	
lrd	
IRWrite	
PCWrite	
PCWriteCond	1

Control Signal (2 bit)	Value
ALUOp	01
ALUSrcB	00
PCSource	01

Multicycle datapath

Step 4: Memory access or R-type instruction completion step

- R-type, memory-access and branch instruction action
- Following tables show control signals for the Load-word instruction

Control Signal (1 bit)	Value
RegWrite	
ALUSrcA	
MemRead	1
MemWrite	
MemtoReg	
IorD	1
IRWrite	
PCWrite	
PCWriteCond	

Multicycle datapath

Step 5: Memory read completion step

- R-type, memory-access and branch instruction action
- Which **Control signals** will/should be used

Control Signal (1 bit)	Value
RegWrite	1
ALUSrcA	
MemRead	
MemWrite	
MemoReg	1
IorD	
IRWrite	
PCWrite	
PCWriteCond	

Multicycle datapath

Summarized overview for all instructions

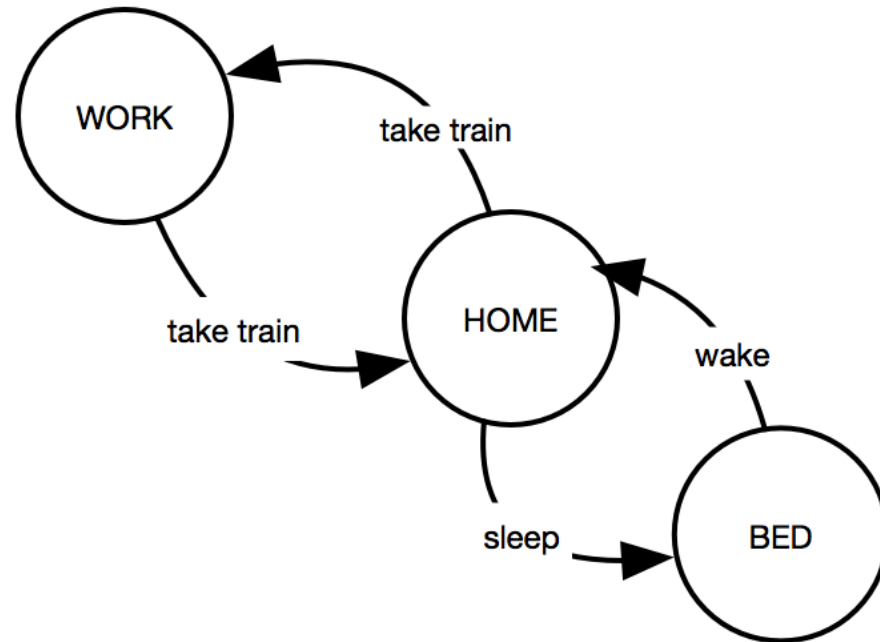
Step name	Action for R-type instructions	Action for memory reference instructions	Action for branches
Instruction fetch	IR $\leftarrow$ Memory[PC] PC $\leftarrow$ PC + 4		
Instruction decode/register fetch	A $\leftarrow$ Reg [IR[19:15]] B $\leftarrow$ Reg [IR[24:20]] ALUOut $\leftarrow$ PC + immediate		
Execution, address computation, branch/jump completion	ALUOut $\leftarrow$ A op B	ALUOut $\leftarrow$ A + immediate	if (A == B) PC $\leftarrow$ ALUOut
Memory access or R-type completion	Reg [IR[11:7]] $\leftarrow$ ALUOut	Load: MDR $\leftarrow$ Memory[ALUOut] or Store: Memory [ALUOut] $\leftarrow$ B	
Memory read completion		Load: Reg[IR[11:7]] $\leftarrow$ MDR	

Multicycle Control

- Let us design and implement the control unit for multicycle processor
- Multicycle control is different from the control unit of single-cycle processor
- Single-cycle control can be designed using only the truth tables
- A Multicycle control would need a finite-state machine as well

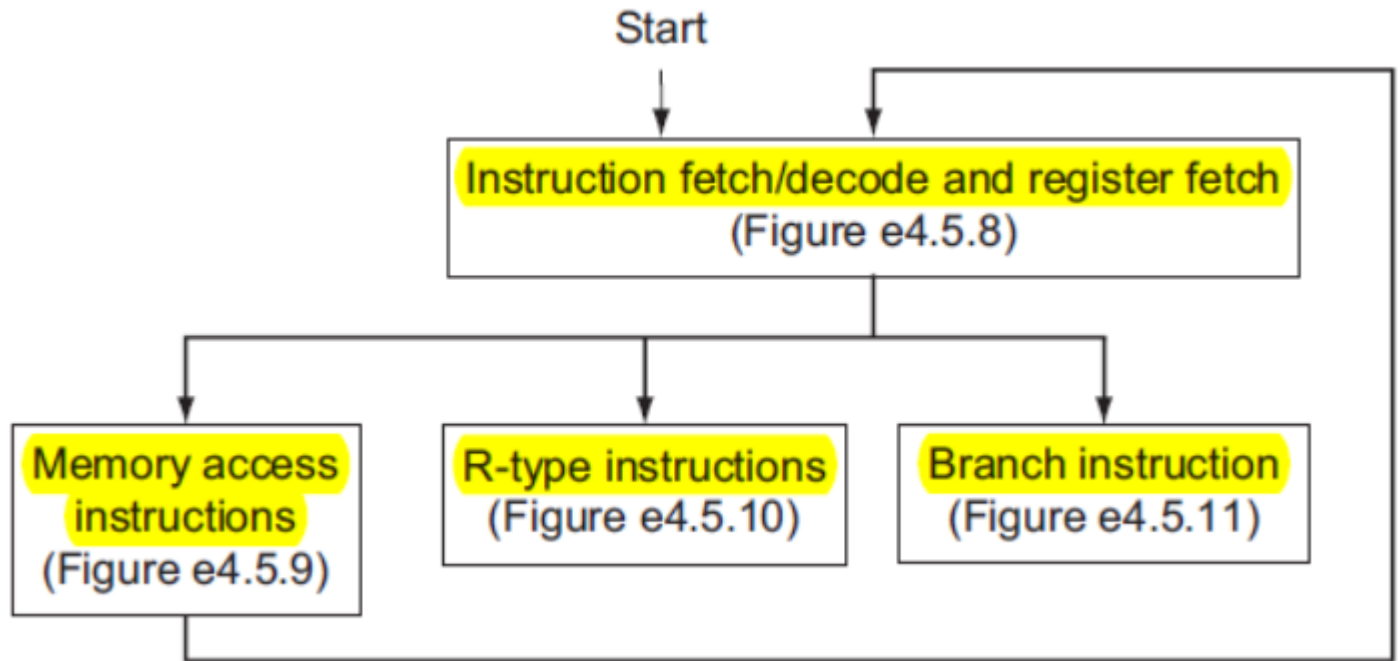
Multicycle Control

■ Finite State Machine



Multicycle Control

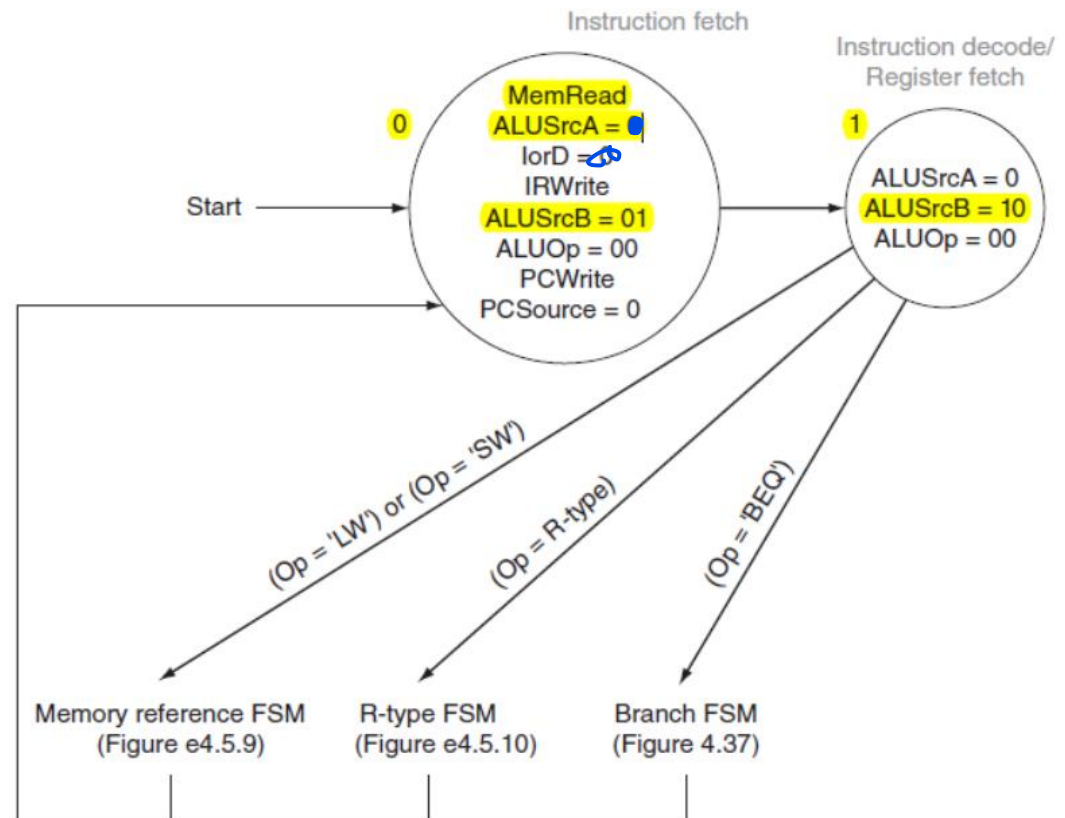
- The finite-state control essentially corresponds to the five steps of an instruction execution, the first two being common



Multicycle Control

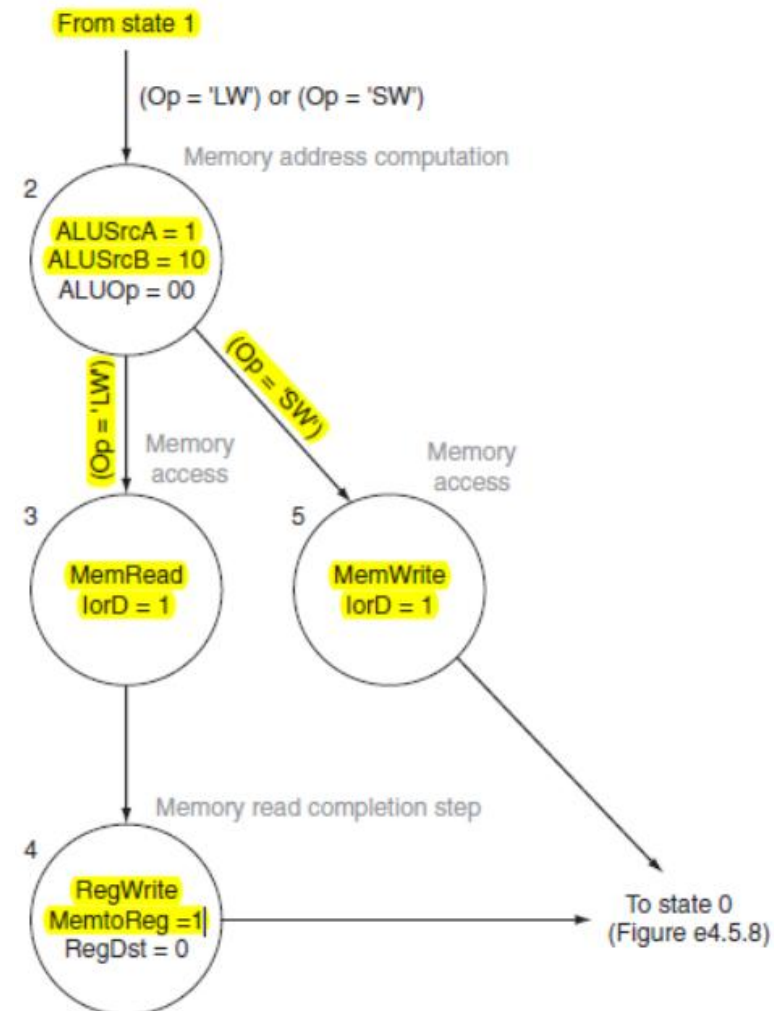
- FSM for multicycle processor control in **first two common steps** in an instruction execution

This process of branching to different states depending on the instruction is called **decoding**



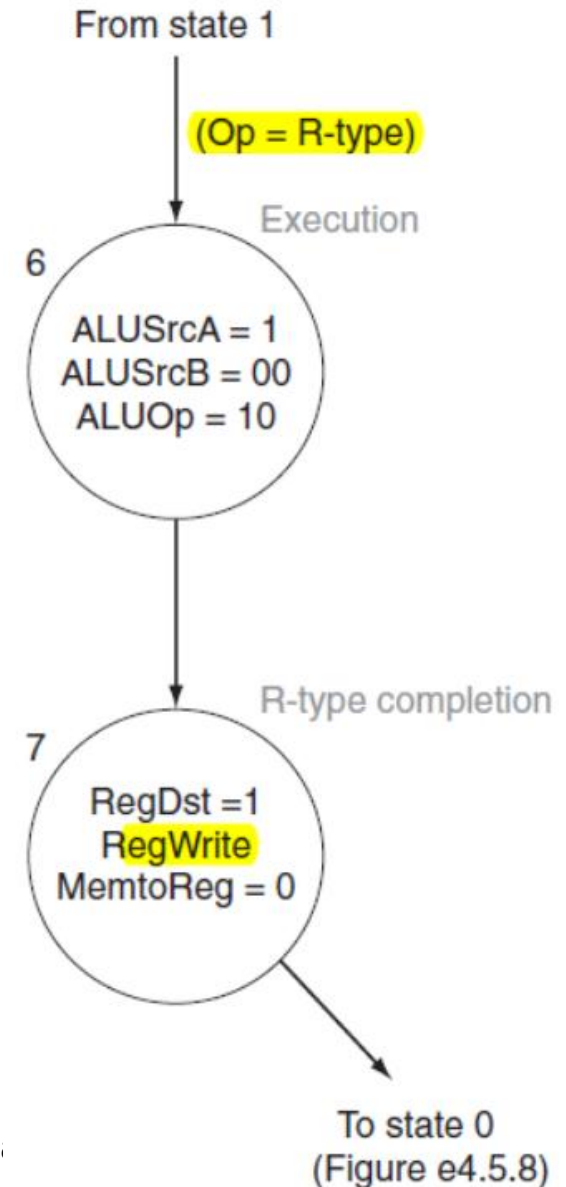
Multicycle Control

- FSM for multicycle processor control in **memory-access** instruction execution



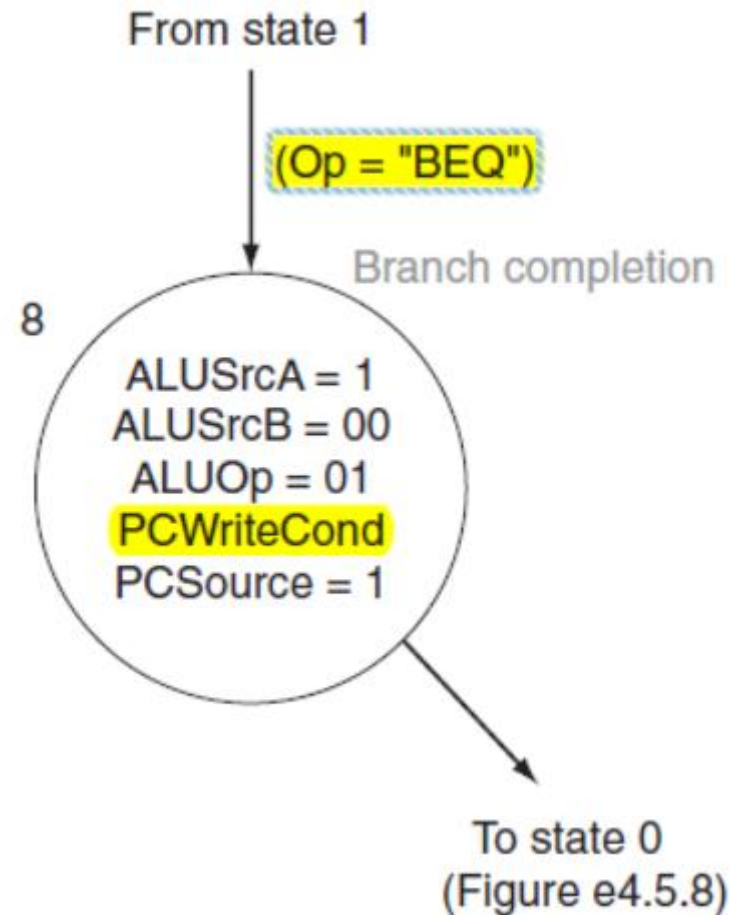
Multicycle Control

- FSM for multicycle processor control in R-type instruction execution



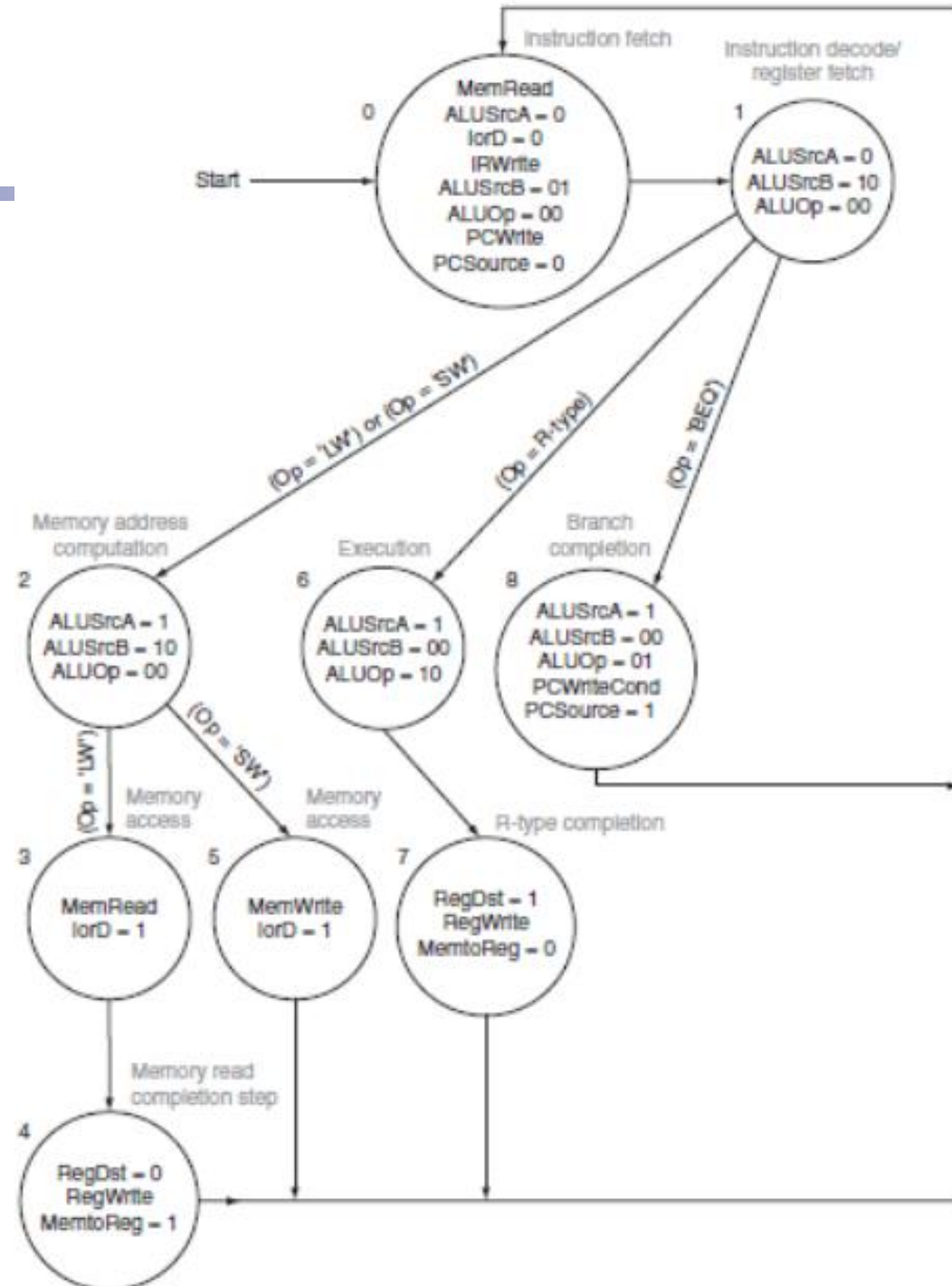
Multicycle Control

- FSM for multicycle processor control in Branch instruction execution



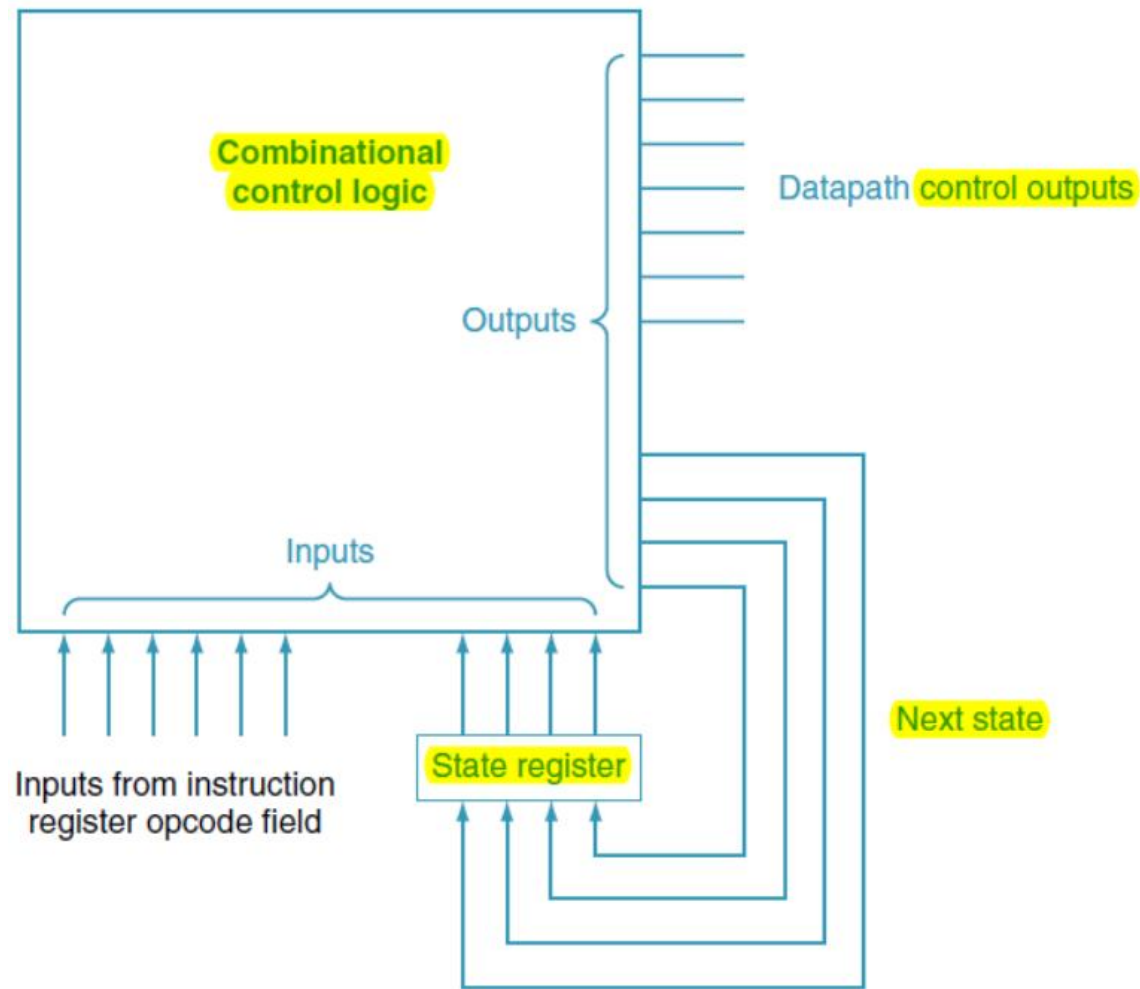
Multicycle Control

- Combined FSM for multicycle processor control in all instructions



Multicycle Control

- Hardware implementation of FSM using Registers and Combinational Logic



Multicycle Control

- Impact on Performance
- Impact of Software via instruction count and CPI
- Conclusion: Programmers seeking to improve performance must understand the role of CPI and the factors that affect it.

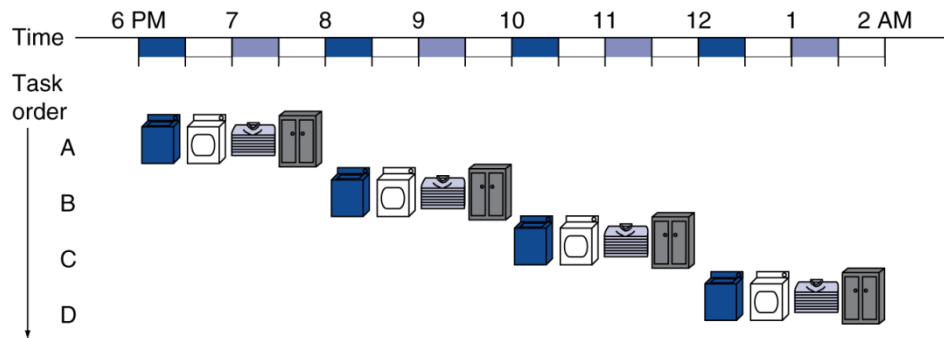
Chapter Outline

Chapter 4 – The Processor

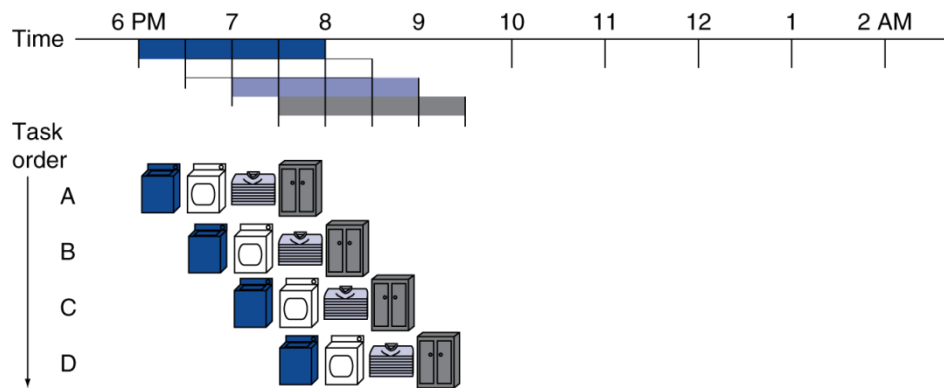
- 4.1 Introduction
- 4.2 Logic Design Conventions
- 4.3 Building a Datapath
- 4.4 A Simple Implementation Scheme
- 4.5 Multicycle Implementation
- 4.6 An Overview of Pipelining
- 4.7 Pipelined Datapath and Control
- 4.8 Data Hazards: Forwarding versus Stalling
- 4.9 Control Hazards
- 4.10 onwards (self-reading)

Pipelining Analogy

- **Pipelined laundry**: overlapping execution
 - **Parallelism** improves performance



- **Four loads:**
 - **Speedup?**



- **Twenty loads:**
 - **Speedup?**

RISC-V Pipeline

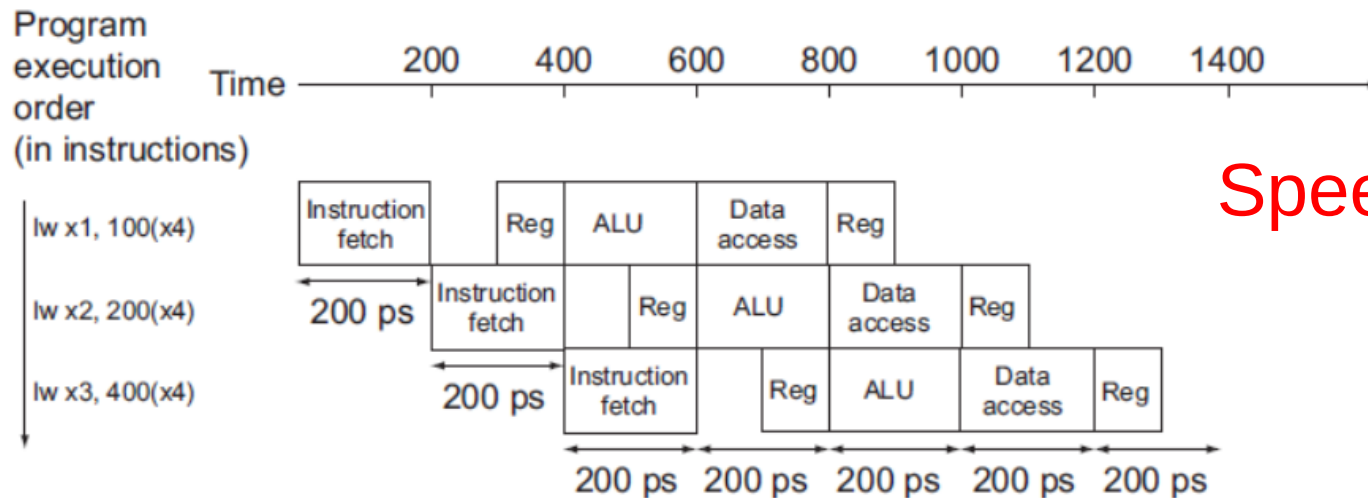
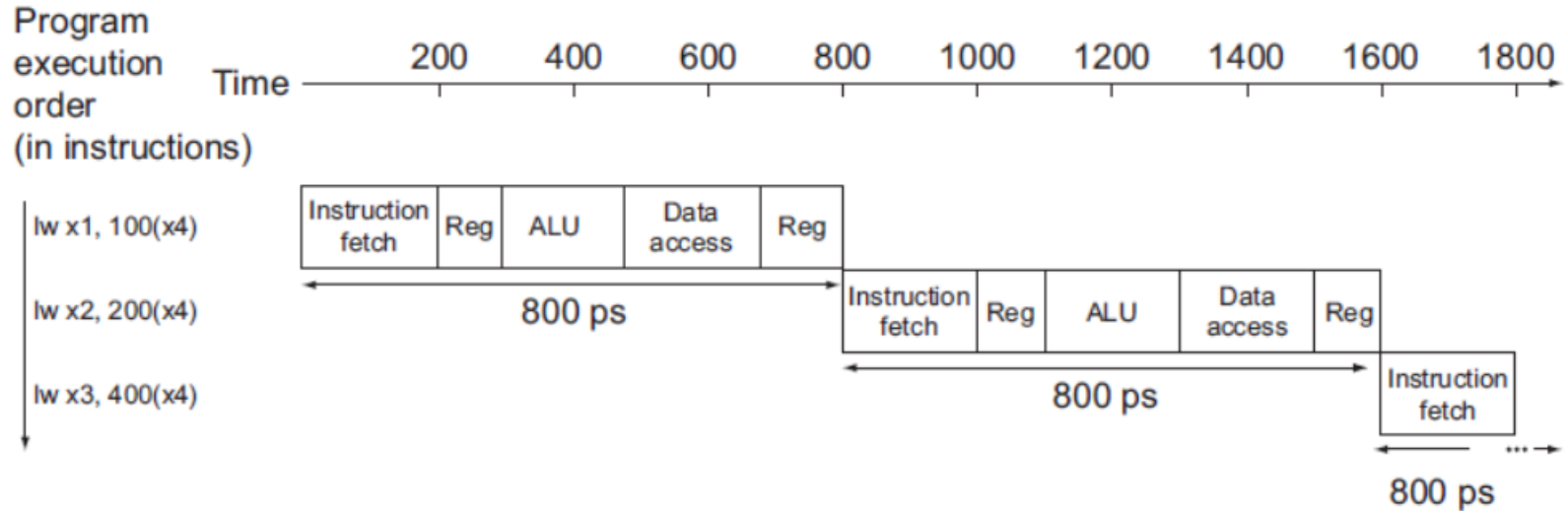
- Five stages, one step per stage
 1. IF: Instruction fetch from memory
 2. ID: Instruction decode & register read
 3. EX: Execute operation or calculate address
 4. MEM: Access memory operand (if necessary)
 5. WB: Write result back to register (if necessary)

Pipeline Performance

- Assume time for stages is
 - 100ps for register read or write
 - 200ps for other stages
- Compare pipelined datapath with single-cycle datapath (lw, sw, R-format, beq)
- Assumptions made?

Instr	Instr fetch	Register read	ALU op	Memory access	Register write	Total time
lw	200ps	100 ps	200ps	200ps	100 ps	800ps
sw	200ps	100 ps	200ps	200ps		700ps
R-format	200ps	100 ps	200ps		100 ps	600ps
beq	200ps	100 ps	200ps			500ps

Pipeline Performance



Speedup?

Pipeline Speedup

- Time between instructions assuming all stages are balanced
- If not balanced, speedup is less
- Pipelining improves performance by *increasing instruction throughput, in contrast to decreasing the execution time of an instruction*

Pipelining and ISA Design

- RISC-V ISA designed for pipelining
 - All instructions are 32-bits
 - Easier to fetch and decode in one cycle
 - c.f. x86: 1- to 17-byte instructions
 - Few and regular instruction formats
 - Can decode and read registers in one step
 - ALU does not need memory operands
 - Can calculate address in 3rd stage, access memory in 4th stage
 - Saves us from a long pipeline!

Hazards

- **Hazards** are situations that prevent starting the next instruction in the next cycle
- Three types:
 1. Structural hazards
 2. Data hazards
 3. Control hazards

Structural Hazards

- **Conflict** for use of a resource (laundry analogy)
- In RISC-V pipeline with a single memory
 - Load/store requires data access
 - **Instruction fetch would have to *stall*** for that cycle – see previous example
 - Would cause a pipeline “bubble”
- Hence, pipelined datapaths require separate instruction/data memories/caches